I 320D · Data Science for Biomedical Informatics · Spring 2026

Step 4: Feature Engineering

Expanded Reference · Challenge Questions with Full Answers

University of Texas at Austin · School of Information

What this document contains

This cheatsheet expands Step 4 of the Project 4 notebook into a complete standalone reference covering:

- Full step-by-step feature engineering instructions: rolling mean/std, delta features, and the groupby-per-patient pattern
- Q17: Rolling window length optimisation — grid-search code, expected AUROC curve shape, and clinical interpretation of the optimum
- Q18: Heart rate variability (HRV) metrics — SDNN, RMSSD, pNN50, and how to compute them from hourly ICU data
- Q19: VIF (Variance Inflation Factor) analysis — implementation, interpretation, and when multicollinearity matters for LR vs LSTM
- Q20: Shock Index as a domain-knowledge feature — implementation, AUROC comparison, and the implicit vs explicit learning debate
- Self-assessment checklist with exam-weight markers

PART 1 — Step-by-Step Feature Engineering Instructions

1. Compute rolling mean and standard deviation (6-hour window)

Apply within each patient's group. Never roll across patient boundaries.

```
WINDOW = 6  # hours

def engineer_features(grp):
    grp = grp.sort_values('Hour')
    for col in FEATURE_COLS:
        s = grp[col]
        grp[f'{col}_roll_mean'] = s.rolling(WINDOW, min_periods=1).mean()
        grp[f'{col}_roll_std']  = s.rolling(WINDOW,
min_periods=1).std().fillna(0)
        grp[f'{col}_delta3']    = s.diff(3).fillna(0)
    return grp

df_feat = (df_imp.groupby('PatientID', group_keys=False)
                 .apply(engineer_features))

ALL_FEATURES = [c for c in df_feat.columns
                if c not in ('PatientID','Hour','SepsisLabel')]
print(f'Total features after engineering: {len(ALL_FEATURES)}')
# Expected: ~120 (40 raw x 3: raw + roll_mean + roll_std + delta3)
```

2. Verify no cross-patient contamination

```
# Quick sanity check: row 0 of each patient should have roll_std = 0
# (only 1 observation, std undefined -> filled with 0)
first_rows = df_feat.groupby('PatientID').first()
bad = (first_rows[[c for c in df_feat.columns if c.endswith('_roll_std')]] !=
0).any().any()
print('Cross-patient contamination detected:', bad)
# Should print: False

# Also verify Hour 0 delta3 = 0 (no prior values to subtract from)
first_rows_hr = df_feat[df_feat['Hour'] == 0]
delta_cols = [c for c in df_feat.columns if c.endswith('_delta3')]
print('Hour-0 deltas all zero:', (first_rows_hr[delta_cols] ==
0).all().all())
```

3. Understand the clinical meaning of each engineered feature

Feature	Formula	Clinical concept encoded	Why it matters for sepsis
HR_roll_mean	mean(HR, last 6h)	Sustained tachycardia	A single elevated HR may be anxiety; persistently elevated HR is compensatory response
HR_roll_std	std(HR, last 6h)	HR variability (crude HRV)	Low HRV in sepsis reflects loss of autonomic regulation — see Q18

Lactate_delta3	Lactate[t] - Lactate[t-3]	Lactate trajectory (clearance or accumulation)	Positive delta = worsening perfusion; negative delta = responding to resuscitation
SBP_delta3	SBP[t] - SBP[t-3]	Rate of blood pressure decline	Rapid SBP drop signals haemodynamic deterioration before absolute threshold is reached
Resp_roll_mean	mean(Resp, last 6h)	Sustained tachypnoea	Compensatory hyperventilation for metabolic acidosis in sepsis
O2Sat_roll_std	std(O2Sat, last 6h)	SpO2 instability	Fluctuating SpO2 indicates respiratory compromise even when mean is acceptable

PART 2 — Challenge Questions: Full Answers

Q17 Rolling window length: finding the optimal look-back horizon for sepsis prediction

HINT *The notebook uses a 6-hour rolling window. Try 3-hour and 12-hour windows. Plot AUROC vs window length. Is there a consistent optimum? What clinical phenomenon might explain the shape of this curve?*

✓ THE SHORT ANSWER

The AUROC vs window length curve typically follows an **inverted-U shape** with a peak somewhere between 4 and 8 hours. Short windows (1–2 hours) capture noise rather than trends. Long windows (> 12 hours) dilute the acute pre-onset signal with stable baseline hours from earlier in the admission. The optimal window corresponds to the **physiological time constant of sepsis deterioration** — roughly the period over which the compensatory mechanisms (tachycardia, tachypnoea, vasoconstriction) evolve before failing. This is approximately 4–8 hours for most sepsis presentations, which is why the 6-hour window is the clinical intuition-backed default.

✓ WHAT THE CURVE SHAPE TELLS YOU — CLINICAL INTERPRETATION

```
Expected AUROC vs rolling window length (illustrative):

AUROC
0.870 |                    *  *
0.865 |               *              *
0.860 |          *                        *
0.855 |     *                                  *
0.850 |  *                                          *
0.845 |
      +----+----+----+----+----+----+----+----+----+---> Window (hours)
           1    2    3    4    6    8   12   16   24

LEFT SIDE (short windows -- 1 to 3h):
-> Too few observations per window; std is noisy
-> Delta features measure moment-to-moment variation, not trend
-> Model cannot distinguish signal from sensor noise

PEAK (4 to 8h):
-> Captures the compensatory physiological response (tachycardia,
   tachypnoea, early hypotension) as it builds over hours
-> Aligns with the early goal-directed therapy 6-hour resuscitation window
-> Most sepsis presentations show detectable trajectory change in this range

RIGHT SIDE (long windows -- > 12h):
-> Stable 'healthy' baseline hours from early admission dilute the
   acute pre-onset signal
-> roll_mean of HR over 24h includes many normal-HR hours before
   deterioration, dragging the mean down toward baseline
-> Temporal resolution is lost: the model cannot distinguish recent
   from remote physiology
```

✓ COMPLETE GRID-SEARCH CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model  import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics       import roc_auc_score
from sklearn.utils         import resample

WINDOWS_TO_TRY = [1, 2, 3, 4, 6, 8, 12, 16, 24]
N_BOOT = 200   # bootstrap samples for confidence intervals
FEATURE_COLS = [c for c in df_imp.columns
                if c not in ('PatientID','Hour','SepsisLabel')]

results = []

for W in WINDOWS_TO_TRY:
    print(f'Window = {W}h ...', end=' ', flush=True)

    # —— Engineer features with this window ——————————————————————————
    def make_features(grp, window=W):
        grp = grp.sort_values('Hour')
        for col in FEATURE_COLS:
            s = grp[col]
            grp[f'{col}_rm']  = s.rolling(window, min_periods=1).mean()
            grp[f'{col}_rs']  = s.rolling(window,
min_periods=1).std().fillna(0)
            grp[f'{col}_d3']  = s.diff(3).fillna(0)
        return grp

    df_w_tr = (df_train.groupby('PatientID', group_keys=False)
                       .apply(make_features))
    df_w_te = (df_test.groupby('PatientID', group_keys=False)
                      .apply(make_features))

    feat_cols = [c for c in df_w_tr.columns
                 if c not in ('PatientID','Hour','SepsisLabel')]

    scaler = StandardScaler()
    X_tr   = scaler.fit_transform(df_w_tr[feat_cols].fillna(0))
    X_te   = scaler.transform(df_w_te[feat_cols].fillna(0))
    y_tr   = df_w_tr['SepsisLabel'].values
    y_te   = df_w_te['SepsisLabel'].values

    lr = LogisticRegression(max_iter=500, class_weight='balanced',
                            C=0.1, random_state=42)
    lr.fit(X_tr, y_tr)
    probs      = lr.predict_proba(X_te)[:,1]
    point_auroc = roc_auc_score(y_te, probs)

    # Bootstrap CI
    boots = [roc_auc_score(y_te[idx], probs[idx])
             for idx in [resample(range(len(y_te)), random_state=i)
                         for i in range(N_BOOT)]]
    lo, hi = np.percentile(boots, [2.5, 97.5])
    results.append({'window': W, 'auroc': point_auroc, 'lo': lo, 'hi': hi})
    print(f'AUROC={point_auroc:.4f}  [{lo:.3f}, {hi:.3f}]')

res_df = pd.DataFrame(results)
```

```
# ── Plot with CI bands ──────────────────────────────────────────────
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(res_df.window, res_df.auroc, 'o-', color='#2C5F8A', lw=2, ms=7)
ax.fill_between(res_df.window, res_df.lo, res_df.hi,
                color='#2C5F8A', alpha=0.18, label='95% CI (bootstrap)')
ax.axvline(6, color='#BF4F00', linestyle='--', lw=1.5,
           label='Default window (6h)')
best = res_df.loc[res_df.auroc.idxmax()]
ax.axvline(best.window, color='#1A6B3C', linestyle=':', lw=1.5,
           label=f'Optimal window ({int(best.window)}h,
AUROC={best.auroc:.3f})')
ax.set_xlabel('Rolling window length (hours)')
ax.set_ylabel('Test AUROC')
ax.set_title('AUROC vs Rolling Window Length\n(LR baseline with forward-
filled imputation)')
ax.legend()
ax.set_xticks(WINDOWS_TO_TRY)
plt.tight_layout()
plt.savefig('window_auroc_curve.png', dpi=120, bbox_inches='tight')
plt.show()
```

✓ EXPECTED RESULTS AND INTERPRETATION TABLE

Window	Expected AUROC (relative to 6h baseline)	Clinical phenomenon driving the change	Action
1h	−2 to −3 AUROC points	Too few samples for meaningful rolling stats; std of 1 sample = 0 by definition	Discard — noise dominates signal at hourly resolution
2h	−1.5 to −2 AUROC points	Some trend capture but insufficient samples for stable std	Discard
3h	−0.5 to −1 AUROC point	Captures rapid deterioration patterns; misses slower-onset presentations	Consider for patients with rapid deterioration subgroup analysis
4h	±0 to +0.3 AUROC points	Captures early compensatory tachycardia and tachypnoea (onset of SIRS response)	Strong candidate; test against 6h
6h (default)	Baseline	Aligns with EGDT 6-hour resuscitation window; most presentations detectable	Use as default; well-validated clinically
8h	±0 to +0.3 AUROC points	Captures slower-onset presentations; slight dilution of acute signal	Test; may outperform 6h on older patient subgroup
12h	−0.2 to −0.8 AUROC points	Baseline physiology from early admission dilutes acute pre-onset signal	Not recommended for snapshot LR; may still help LSTM (temporal

			architecture handles it differently)
24h	−1 to −2 AUROC points	Severe dilution; 18h of stable data averages out the 6h deterioration signal	Discard for LR; marginally useful for LSTM only

Key exam takeaway

The optimal window length is a hyperparameter that encodes a clinical assumption about the relevant physiological time scale. The inverted-U curve exists because there is a minimum window for statistical stability and a maximum window beyond which baseline physiology dominates.

Important nuance: the optimal window may differ by model type. For LR (which receives pre-computed rolling features), shorter windows reduce feature quality. For LSTM (which processes raw hourly values), the SEQ_LEN hyperparameter plays the equivalent role — and the LSTM can learn its own effective window length from data.

Report the window optimisation curve in your paper. Reviewers will ask why you chose 6 hours, and 'it is the default' is not a sufficient answer. 'Cross-validated AUROC peaks at 6 hours with 95% CIs that exclude shorter and longer windows' is.

Q18 Heart rate variability (HRV): computing clinically valid metrics from hourly ICU data

HINT *Heart rate variability (HRV) is a known prognostic marker in sepsis. The standard deviation of HR over 6 hours is a crude proxy. How would you compute a more clinically valid HRV metric from hourly data?*

✓ THE SHORT ANSWER

True HRV is computed from the millisecond-resolution RR interval (time between heartbeats). From hourly ICU data, you cannot compute gold-standard HRV metrics because the temporal resolution is too low (1 sample per hour vs 60–3600 samples per minute for clinical ECG). However, you can compute **surrogate HRV metrics** that capture the same biological signal: autonomic nervous system activity. The most useful surrogates from hourly data are (1) SDNN-proxy (rolling SD of HR over 6h), (2) successive difference metrics, and (3) the coefficient of variation. Each captures a different aspect of autonomic tone, and their decline in sepsis reflects the transition from compensated to decompensated organ dysfunction.

✓ CLINICAL BACKGROUND: WHY HRV MATTERS IN SEPSIS

In healthy individuals, the autonomic nervous system constantly modulates heart rate — up with activity, down with rest. This variability reflects the dynamic balance between sympathetic (fight-or-flight) and parasympathetic (rest) tone. In sepsis, the systemic inflammatory response progressively impairs autonomic regulation:

```
  Early sepsis:  Sympathetic activation -> tachycardia + preserved variability
                 HRV may actually INCREASE transiently (higher sympathetic tone)

  Established:   Autonomic decoupling begins -> HR rises but variability falls
                 Characteristic: high mean HR + low rolling std of HR

  Severe/organ:  Autonomic failure -> HR becomes a fixed 'machine' rhythm
   dysfunction   Very low HRV; HR no longer responds to stimuli
                 Associated with multi-organ failure and mortality

  Clinical use:  HRV monitoring is recommended in sepsis ICU care
                 A falling SDNN is associated with adverse outcomes
                 HeartRate_roll_std == 0 for >3 consecutive hours is a critical
sign
```

✓ HRV METRICS FROM HOURLY DATA: DEFINITIONS AND CODE

Metric	Gold standard definition	Hourly ICU approximation	What it captures
SDNN	Standard deviation of all NN intervals over 24h from ECG	SD of hourly HR values over a rolling window	Overall HRV; reflects combined ANS activity

RMSSD	Root mean square of successive differences between NN intervals	RMSD of successive hourly HR values: sqrt(mean((HR[t]-HR[t-1])^2))	Short-term parasympathetic activity; vagal tone
pNN50	Percentage of consecutive NN intervals differing by > 50ms	Percentage of consecutive hourly HR values differing by > 5 bpm (adjusted threshold)	Parasympathetic dominance marker; immune correlation
CV (Coeff. of Variation)	SD / mean of NN intervals	HR_roll_std / HR_roll_mean (dimensionless)	Normalised variability; useful when comparing across different HR baselines
Range	Max - min of NN interval	Max - min of hourly HR in rolling window	Simple range; robust to single outliers when using trimmed range

✓ COMPLETE IMPLEMENTATION CODE

```
import numpy as np
import pandas as pd

def compute_hrv_features(grp, window=6):
    """
    Compute surrogate HRV metrics from hourly HR values.
    Applied per-patient within groupby.
    """
    grp = grp.sort_values('Hour').copy()
    hr  = grp['HR']

    # ── 1. SDNN proxy: rolling SD of HR ──────────────────────────────
    grp['HR_sdnn_proxy'] = hr.rolling(window, min_periods=2).std().fillna(0)

    # ── 2. RMSSD proxy: root mean square of successive differences ────
    successive_diff_sq = hr.diff(1).pow(2)   # (HR[t] - HR[t-1])^2
    grp['HR_rmssd_proxy'] = (successive_diff_sq
                             .rolling(window, min_periods=2)
                             .mean()
                             .apply(np.sqrt)
                             .fillna(0))

    # ── 3. pNN50 proxy: fraction of consecutive HR changes > 5 bpm ────
    # 5 bpm threshold adapted from 50ms original (1 bpm ≈ 10ms at HR 60-100)
    large_changes = hr.diff(1).abs() > 5
    grp['HR_pnn50_proxy'] = (large_changes
                             .rolling(window, min_periods=2)
                             .mean()
                             .fillna(0))

    # ── 4. Coefficient of variation ──────────────────────────────────
    roll_mean = hr.rolling(window, min_periods=2).mean()
    roll_std  = hr.rolling(window, min_periods=2).std()
    grp['HR_cv'] = (roll_std / roll_mean.replace(0, np.nan)).fillna(0)
```

```
    # ── 5. Trimmed range (robust to single outliers) ──────────────────
    def trimmed_range(x):
        if len(x) < 3:
            return 0
        sorted_x = sorted(x.dropna())
        trim = max(1, len(sorted_x) // 5)  # trim bottom/top 20%
        return sorted_x[-trim] - sorted_x[trim] if len(sorted_x) > 2*trim
else 0

    grp['HR_trimmed_range'] = (hr
                               .rolling(window, min_periods=3)
                               .apply(trimmed_range, raw=False)
                               .fillna(0))

    return grp

print('Computing HRV features...')
df_hrv = (df_imp.groupby('PatientID', group_keys=False)
                .apply(compute_hrv_features))

# ── Validate: sepsis vs non-sepsis HRV ──────────────────────────────
hrv_cols = ['HR_sdnn_proxy','HR_rmssd_proxy','HR_pnn50_proxy','HR_cv']
print('Mean HRV metrics by sepsis status (at hours up to 6h before onset):')
for col in hrv_cols:
    m0 = df_hrv[df_hrv.SepsisLabel==0][col].mean()
    m1 = df_hrv[df_hrv.SepsisLabel==1][col].mean()
    print(f'  {col:<22}  Non-sepsis: {m0:.3f}   Sepsis: {m1:.3f}   Ratio:
{m1/m0:.2f}')
# Expected: sepsis patients show LOWER HRV metrics (autonomic dysfunction)
```

✓ EXPECTED VALIDATION FINDINGS

Metric	Non-sepsis mean (approx)	Sepsis mean (approx)	Direction	Interpretation
HR_sdnn_proxy	8 – 12 bpm	5 – 8 bpm	Sepsis LOWER	Reduced HR variability in sepsis — autonomic decoupling
HR_rmssd_proxy	6 – 10	3 – 6	Sepsis LOWER	Reduced short-term vagal activity in sepsis — parasympathetic withdrawal
HR_pnn50_proxy	0.25 – 0.40	0.10 – 0.20	Sepsis LOWER	Fewer large beat-to-beat changes — 'monotonous tachycardia' pattern

HR_cv	0.08 – 0.12	0.04 – 0.07	Sepsis LOWER	Normalised variability decline even when absolute HR is elevated

Limitations of hourly HRV proxies	When hourly HRV IS sufficient
Temporal resolution	True HRV requires beat-to-beat data (millisecond resolution). Hourly HR is 3,600x lower resolution. Fine-grained frequency domain features (LF/HF ratio, spectral analysis) are not computable from hourly data.
Clinical validation	Hourly HRV proxies are not validated against the standard HRV literature (which uses 5-minute ECG segments). Comparisons to published sepsis HRV studies must be made cautiously.
Medication confounding	Beta-blockers reduce HR variability artificially; sedation suppresses autonomic activity. These effects are indistinguishable from pathological HRV reduction.

Key exam takeaway

The best answer to 'how do you compute a more clinically valid HRV metric from hourly data?' has three parts: (1) explain why true HRV requires beat-to-beat resolution and why hourly data is insufficient for standard metrics; (2) propose RMSSD proxy and CV as the most clinically interpretable approximations; (3) validate empirically that sepsis patients show lower HRV in the 6 hours before onset.

The most important finding: HR_rmssd_proxy < 3 bpm sustained for 3+ hours is a potential alert criterion. A patient with high sustained HR (> 95) AND very low RMSSD proxy (< 2) represents the pathological 'fixed tachycardia' pattern characteristic of severe sepsis.

Q19 VIF analysis: does multicollinearity matter for LR and LSTM?

> **HINT** *Some features have high collinearity after engineering (e.g. HR_roll_mean and HR are correlated). Run a VIF (Variance Inflation Factor) analysis. Does multicollinearity affect logistic regression performance? Does it affect the LSTM?*

✓ THE SHORT ANSWER

For AUROC: multicollinearity has little effect on LR performance. L2 regularisation (the C parameter in scikit-learn) distributes coefficient mass across correlated features, preventing any single one from dominating. AUROC is a ranking metric — the model's ability to rank patients by risk is not harmed by correlated features, just its coefficient interpretability. **For SHAP interpretability: multicollinearity is a serious problem.** SHAP values for correlated features are distributed arbitrarily between them — the importance of HR vs HR_roll_mean is split in ways that depend on the random sample order and cannot be relied upon. For LSTM: multicollinearity in the input layer is not a concern because LSTMs are non-linear and the temporal architecture handles redundant features differently.

✓ WHAT VIF MEASURES

```
  VIF (Variance Inflation Factor) for feature j:

  VIF_j = 1 / (1 - R²_j)

  where R²_j is the R² from regressing feature j on all OTHER features.

  Interpretation:
  VIF = 1.0       : feature j is completely uncorrelated with all other features
  VIF = 1 to 5    : mild collinearity; generally acceptable
  VIF = 5 to 10   : moderate collinearity; interpret coefficients cautiously
  VIF > 10        : severe collinearity; coefficients are unreliable for
interpretation
  VIF > 30        : extreme collinearity; feature is nearly a linear combination
                    of others (e.g. HR and HR_roll_mean over 1h will have VIF >
30)

  Rule: VIF does NOT indicate that prediction performance is degraded.
        It indicates that COEFFICIENT INTERPRETATION is unreliable.
```

✓ COMPLETE VIF IMPLEMENTATION CODE

```
import numpy as np
import pandas as pd
from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# VIF is computed on a sample (full dataset is too large for VIF on 120
features)
# Use a random sample of 5000 rows, with at least some sepsis examples
np.random.seed(42)
sample_idx = np.random.choice(len(df_feat), size=min(5000, len(df_feat)),
                              replace=False)
df_sample  = df_feat.iloc[sample_idx][ALL_FEATURES].fillna(0)
```

```
# Standardise before VIF (VIF is scale-invariant but standardising helps
# numerical stability for large feature sets)
sc     = StandardScaler()
X_std  = sc.fit_transform(df_sample)
X_df   = pd.DataFrame(X_std, columns=ALL_FEATURES)

print('Computing VIF for all features (this may take 2-3 minutes)...')
vif_data = pd.DataFrame({
    'Feature': ALL_FEATURES,
    'VIF':     [variance_inflation_factor(X_df.values, i)
                for i in range(X_df.shape[1])]
}).sort_values('VIF', ascending=False)

print('\nTop 20 highest VIF features:')
print(vif_data.head(20).to_string(index=False))
print(f'\nFeatures with VIF > 10: {(vif_data.VIF > 10).sum()}')
print(f'Features with VIF > 30: {(vif_data.VIF > 30).sum()}')

# ── Visualise VIF distribution ──────────────────────────────────────────
fig, ax = plt.subplots(figsize=(12, 5))
vif_plot = vif_data.head(30)  # top 30 only for readability
colors   = ['#8B1A1A' if v > 30 else '#8B5E00' if v > 10 else '#2C5F8A'
            for v in vif_plot.VIF]
ax.barh(range(len(vif_plot)), vif_plot.VIF, color=colors)
ax.set_yticks(range(len(vif_plot)))
ax.set_yticklabels(vif_plot.Feature, fontsize=8)
ax.axvline(10, color='#8B5E00', linestyle='--', lw=1.5, label='VIF=10
threshold')
ax.axvline(30, color='#8B1A1A', linestyle='--', lw=1.5, label='VIF=30
threshold')
ax.set_xlabel('Variance Inflation Factor')
ax.set_title('Top 30 Features by VIF (red = extreme, orange = high, blue =
acceptable)')
ax.legend()
plt.tight_layout()
plt.savefig('vif_analysis.png', dpi=120, bbox_inches='tight')
plt.show()

# ── Test: does dropping high-VIF features change AUROC? ─────────────────
from sklearn.linear_model import LogisticRegression
from sklearn.metrics      import roc_auc_score

low_vif_features = vif_data[vif_data.VIF <= 10]['Feature'].tolist()
print(f'\nFeatures with VIF <= 10: {len(low_vif_features)}')

for label, feats in [('All features', ALL_FEATURES),
                     ('VIF <= 10 only', low_vif_features)]:
    X_tr = StandardScaler().fit_transform(
               df_train[feats].fillna(0))
    X_te = StandardScaler().fit(df_train[feats].fillna(0)).transform(
               df_test[feats].fillna(0))
    y_tr = df_train['SepsisLabel'].values
    y_te = df_test['SepsisLabel'].values
    lr = LogisticRegression(max_iter=500, class_weight='balanced',
                            C=0.1, random_state=42)
    lr.fit(X_tr, y_tr)
    auroc = roc_auc_score(y_te, lr.predict_proba(X_te)[:,1])
    print(f'{label:<22}  n_features={len(feats):3d}  AUROC={auroc:.4f}')
```

✓ EXPECTED FINDINGS AND INTERPRETATION

Feature group	Expected VIF	Why	Impact on LR AUROC	Impact on SHAP interpretation
HR and HR_roll_mean (1h window)	VIF > 50	Nearly identical: mean of 1 value ≈ the value itself	Negligible — L2 reg distributes weight	Severe: SHAP splits importance arbitrarily between identical features
HR_roll_mean (6h) and HR_roll_mean (3h)	VIF 15–30	Highly correlated moving averages of the same series	Negligible	Moderate: importance shared but features capture slightly different signals
SBP, MAP, DBP (blood pressure family)	VIF 10–25	MAP = (SBP + 2×DBP)/3 by definition — mathematical linear relationship	Negligible	Moderate: SHAP for MAP partly captures SBP and DBP signal
HR_roll_std and HR_rmssd_proxy	VIF 8–15	Both measure HR variability over a window	Negligible	Low-moderate: they capture overlapping but distinct signals (std vs successive differences)
Raw features (HR, Resp, Temp, etc.)	VIF 2–5	Physiologically correlated but not linearly dependent	None	Low: independent enough for interpretable SHAP values

✓ LR VS LSTM: HOW EACH HANDLES MULTICOLLINEARITY

Logistic Regression	LSTM

Effect of high VIF on prediction	None on AUROC with L2 regularisation. The penalty shrinks all correlated feature coefficients together.
Effect on interpretation	Severe. SHAP values for HR and HR_roll_mean cannot be trusted individually when VIF > 20. The importance is split arbitrarily based on numerical optimisation details.
Recommended action	For interpretation: drop one of each correlated pair (prefer roll_mean over raw for snapshot models). For prediction: keep all features; L2 handles it.
What removing high-VIF features does to AUROC	Typically < 0.005 AUROC change. This confirms prediction robustness.

Key exam takeaway

The answer has a critical distinction: VIF measures whether coefficients are interpretable, NOT whether the model predicts well. An AUROC comparison between all-features and VIF-filtered features that shows < 0.005 difference confirms that multicollinearity is a purely interpretability problem, not a performance problem.

The practical recommendation: for a paper reporting SHAP feature importance, use a VIF-filtered feature set (drop one of each pair with VIF > 20) for the SHAP analysis while keeping the full feature set for the actual model. Report both the full-feature AUROC and the SHAP analysis on the reduced set.

Q20 Shock Index: explicit domain-knowledge feature vs implicit model learning

> **HINT** *Design a 'shock index' feature (HR / SBP). Does adding this domain-knowledge feature improve AUROC compared to letting the model learn it implicitly?*

✓ THE SHORT ANSWER

The Shock Index (SI = HR / SBP) will typically produce a **small but measurable AUROC improvement for LR and no improvement for LSTM.** The reason is architectural: logistic regression is linear and cannot learn non-linear ratios implicitly — it can approximate HR/SBP only through a high-order polynomial expansion of HR and SBP terms. The LSTM, being non-linear, can approximate ratio relationships through its gated activations. Therefore, explicit Shock Index matters more for simpler models and less for architecturally richer ones. The broader lesson: domain-knowledge features are a form of **inductive bias injection** that compensates for model architecture limitations.

✓ THE CLINICAL BASIS FOR SHOCK INDEX

```
  Shock Index = Heart Rate (bpm) / Systolic Blood Pressure (mmHg)

  Normal range:  0.5 - 0.7
  Alert threshold: SI > 0.9  (compensated shock)
  Critical threshold: SI > 1.0 (decompensated shock)
  Severe threshold: SI > 1.4  (high mortality risk)

  Why the ratio matters:
  HR = 80, SBP = 120  -> SI = 0.67  (normal)
  HR = 100, SBP = 100 -> SI = 1.00  (compensated; BP maintained but only via
tachycardia)
  HR = 120, SBP = 80  -> SI = 1.50  (decompensated; cardiovascular compromise)

  The key insight: two patients with HR=100 look identical by HR alone.
  But SBP=100 (SI=1.0) is very different from SBP=140 (SI=0.71).
  The ratio captures the HAEMODYNAMIC RESERVE: how much is the heart
  working relative to what pressure it is generating?

  Clinical validation: SI > 0.9 predicts septic shock (AUC 0.73-0.81)
  in multiple emergency medicine studies. SI > 1.0 predicts ICU admission.
```

✓ WHY LR CANNOT LEARN SHOCK INDEX IMPLICITLY

```
  Logistic regression models the log-odds as a LINEAR function of features:

  log(P/(1-P)) = b0 + b1*HR + b2*SBP + ...

  To approximate HR/SBP, LR would need terms like:
  HR * (1/SBP)  -- a non-linear interaction

  LR approximation of SI using only HR and SBP terms:
  b1*HR + b2*SBP ≈ SI  only when SBP is approximately constant

  If SBP varies (which it does -- from 70 to 200 in ICU patients),
  the linear approximation fails badly.
```

```
  Example failure: HR=120, SBP=80 (SI=1.5, critical) vs HR=60, SBP=40 (SI=1.5,
also critical)
  LR assigns DIFFERENT probabilities because it adds b1*120+b2*80 vs b1*60+b2*40.
  The actual SI is the same; the LR score is not.

  Explicit SI feature gives LR the pre-computed ratio directly, bypassing
  the linear architecture limitation.
```

✓ COMPLETE IMPLEMENTATION AND COMPARISON CODE

```
import numpy as np
import pandas as pd
from sklearn.linear_model  import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics       import roc_auc_score
from scipy.stats           import ttest_ind

# ── 1. Compute Shock Index and variants ──────────────────────────────────
def add_shock_index_features(df):
    df = df.copy()
    # Basic Shock Index
    df['ShockIndex'] = df['HR'] / df['SBP'].replace(0, np.nan)
    df['ShockIndex'] = df['ShockIndex'].clip(0, 5)  # cap outliers (SBP near
0)

    # Modified Shock Index (uses MAP instead of SBP -- more stable)
    df['ShockIndex_MAP'] = df['HR'] / df['MAP'].replace(0, np.nan)
    df['ShockIndex_MAP'] = df['ShockIndex_MAP'].clip(0, 5)

    # Age-adjusted Shock Index (SIPA): SI * Age -- correlates with mortality
    if 'Age' in df.columns:
        df['SIPA'] = df['ShockIndex'] * df['Age']

    # Rolling Shock Index (trend is more informative than snapshot)
    def add_rolling_si(grp):
        grp = grp.sort_values('Hour')
        grp['ShockIndex_rm'] = grp['ShockIndex'].rolling(6,
min_periods=1).mean()
        grp['ShockIndex_d3'] = grp['ShockIndex'].diff(3).fillna(0)
        return grp

    df = df.groupby('PatientID', group_keys=False).apply(add_rolling_si)
    return df

df_si = add_shock_index_features(df_feat)

# Validate: SI should be higher in sepsis hours
si_sep = df_si[df_si.SepsisLabel==1]['ShockIndex'].mean()
si_nos = df_si[df_si.SepsisLabel==0]['ShockIndex'].mean()
print(f'Mean SI: Sepsis={si_sep:.3f}  Non-sepsis={si_nos:.3f}
Ratio={si_sep/si_nos:.2f}')

# ── 2. Compare AUROC: with and without Shock Index features ──────────────
SI_FEATURES  =
['ShockIndex','ShockIndex_MAP','SIPA','ShockIndex_rm','ShockIndex_d3']
SI_FEATURES  = [f for f in SI_FEATURES if f in df_si.columns]

BASE_FEATURES = [c for c in df_feat.columns
```

```
                 if c not in ('PatientID','Hour','SepsisLabel')]
FULL_FEATURES = BASE_FEATURES + SI_FEATURES

results = {}
for label, feats in [('Without SI', BASE_FEATURES),
                     ('With SI',    FULL_FEATURES)]:
    sc   = StandardScaler()
    X_tr = sc.fit_transform(df_train.reindex(columns=feats).fillna(0))
    X_te = sc.transform(df_test.reindex(columns=feats).fillna(0))
    y_tr = df_train['SepsisLabel'].values
    y_te = df_test['SepsisLabel'].values
    lr   = LogisticRegression(max_iter=1000, class_weight='balanced',
                              C=0.1, random_state=42)
    lr.fit(X_tr, y_tr)
    probs = lr.predict_proba(X_te)[:,1]
    auroc = roc_auc_score(y_te, probs)
    results[label] = {'auroc': auroc, 'probs': probs}
    print(f'{label:<15}  AUROC={auroc:.4f}  n_features={len(feats)}')

delta = results['With SI']['auroc'] - results['Without SI']['auroc']
print(f'\nDelta AUROC from adding Shock Index: {delta:+.4f}')

# ── 3. Statistical test: DeLong test for paired AUROCs ─────────────────
# (approximate: use bootstrap for simplicity)
from sklearn.utils import resample
y_te  = df_test['SepsisLabel'].values
p_no  = results['Without SI']['probs']
p_si  = results['With SI']['probs']

boot_deltas = []
for _ in range(1000):
    idx = resample(range(len(y_te)), random_state=None)
    delta_b = roc_auc_score(y_te[idx], p_si[idx]) - roc_auc_score(y_te[idx],
p_no[idx])
    boot_deltas.append(delta_b)

ci_lo, ci_hi = np.percentile(boot_deltas, [2.5, 97.5])
print(f'Bootstrap 95% CI for delta AUROC: [{ci_lo:+.4f}, {ci_hi:+.4f}]')
sig = 'statistically significant' if ci_lo > 0 else 'not statistically
significant'
print(f'Conclusion: improvement is {sig}')
```

✓ EXPECTED RESULTS AND INTERPRETATION MATRIX

Finding	Expected result	Correct interpretation	Action
LR AUROC with SI > without SI	Delta AUROC +0.003 to +0.012 for LR	LR was unable to learn the HR/SBP ratio implicitly; explicit SI compensates for the linear architecture limitation	Retain SI features in LR model
LR improvement statistically significant	95% CI lower bound > 0 (depends on dataset size)	The improvement is not due to chance; SI encodes information the	Report SI as an important domain-knowledge

		linear model cannot extract otherwise	feature in the paper
LSTM AUROC with SI ≈ without SI	Delta AUROC < 0.003 for LSTM	The LSTM's non-linear gating already approximates ratio relationships; SI is redundant	SI features are optional for LSTM; include for interpretability but not performance
ShockIndex_d3 (rolling delta) > ShockIndex alone in SHAP	ShockIndex_d3 typically more important than point SI	Trajectory of haemodynamic compromise matters more than snapshot value — consistent with Q17 findings	Prioritise rolling SI over point SI in feature set
Mean SI sepsis > non-sepsis	SI ≈ 1.0 in sepsis vs ≈ 0.7 in non-sepsis	Validates clinical literature: SI > 0.9 as sepsis alert threshold holds in this dataset	Use SI > 0.9 as a simple clinical rule benchmark to compare against the model

✓ THE BROADER LESSON: IMPLICIT VS EXPLICIT LEARNING

The Shock Index experiment illustrates a general principle in clinical ML:

Domain-knowledge features are a form of inductive bias: They tell the model 'this combination of features has a known clinical meaning.' When the model architecture is expressive enough to learn the combination anyway (LSTM), the explicit feature is redundant. When the architecture cannot (LR, linear), it is essential.

The correct decision rule: Add explicit domain-knowledge features (SI, A-a gradient, SOFA score components, pulse pressure) when (a) there is strong clinical validation in the literature, (b) the base model is linear or low-capacity, and (c) the feature represents a ratio, product, or threshold that requires non-linear computation. Do not add purely for intuition — validate empirically with an AUROC comparison.

Beware of redundancy inflation: Adding many domain-knowledge features to an already-rich feature set increases dimensionality without proportionally increasing information. With L2 regularisation this is harmless for AUROC but can dilute SHAP interpretability. Track the number of features and their VIF when adding domain-knowledge features.

Key exam takeaway

The three-part answer to 'does Shock Index improve AUROC?': (1) empirically: yes for LR (+0.003 to +0.012), negligible for LSTM; (2) mechanistically: LR cannot learn non-linear ratios implicitly due to its linear architecture; (3) generally: domain-knowledge features compensate for model architecture limitations and are more valuable for simpler models.

The ShockIndex > 0.9 rule is itself a clinical benchmark worth computing: how does it perform as a standalone sepsis alert? If it achieves AUROC 0.73 and your full LR model achieves 0.85, you have demonstrated that your model adds substantial value beyond what any single bedside rule provides.

Self-Assessment Checklist — Step 4

Check each item after completing Step 4. Items marked ★ are likely exam questions.

Feature Engineering Fundamentals

- ☐ I can write the engineer_features function from memory (rolling mean, std, diff(3))
- ☐ ★ I can explain why features must be engineered within patient groups (no cross-patient contamination)
- ☐ I know how to verify no contamination occurred (check Hour 0 roll_std = 0)
- ☐ I can map 4 engineered features to their clinical concepts (e.g. Lactate_delta3 = lactate trajectory)

Q17 — Rolling Window Optimisation

- ☐ ★ I can describe the inverted-U shape of the AUROC vs window length curve and explain both sides
- ☐ I know which clinical phenomenon defines the peak (physiological time constant of sepsis deterioration ~4-8h)
- ☐ I can implement the grid-search with bootstrap CIs in code
- ☐ ★ I know why the optimal window differs between LR (pre-computed features) and LSTM (SEQ_LEN parameter)
- ☐ I know how to justify the choice of 6h in a paper (cross-validated curve, not just 'it is the default')

Q18 — Heart Rate Variability

- ☐ ★ I can explain why true HRV requires beat-to-beat data and why hourly data is insufficient for gold-standard metrics
- ☐ I can define SDNN, RMSSD, and pNN50 in plain English and give their hourly ICU approximations
- ☐ ★ I know the clinical direction: sepsis patients show LOWER HRV (autonomic decoupling)
- ☐ I can implement RMSSD proxy (rolling sqrt(mean(successive differences squared))) in pandas
- ☐ I know the three confounders: beta-blockers, sedation, and cardiac monitoring artefacts

Q19 — VIF and Multicollinearity

- ☐ ★ I can explain what VIF measures (interpretability, NOT prediction performance)
- ☐ I know VIF thresholds: 1-5 acceptable, 5-10 moderate, >10 concerning, >30 extreme
- ☐ ★ I can explain why L2 regularisation protects LR AUROC from multicollinearity
- ☐ I know why SHAP interpretation is unreliable when VIF > 20 between feature pairs ★
- ☐ I can implement VIF using statsmodels variance_inflation_factor
- ☐ I know the recommended action: keep all features for AUROC, use VIF-filtered set for SHAP

Q20 — Shock Index

- ☐ ★ I can compute Shock Index (HR/SBP) and state the clinical thresholds (0.9 alert, 1.0 critical)
- ☐ I can explain why LR benefits from explicit SI more than LSTM (linear vs non-linear architecture)
- ☐ ★ I know the general principle: domain-knowledge features compensate for model architecture limitations
- ☐ I can implement the AUROC comparison with bootstrap CIs for statistical significance
- ☐ I know to also compute ShockIndex_rm and ShockIndex_d3 (trajectory matters more than snapshot) ★

How to use this cheatsheet responsibly

Step 4 questions test both implementation fluency (can you write the code?) and conceptual depth (can you explain why?). For exams, the 'why' is worth more than the 'how'. Items marked ★ are conceptual — practice explaining them in 2-3 sentences without looking at notes.